OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

Course: Object Oriented Programming with C++ (DSA01)
Course Outcome: CO4 – Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers and polymorphism to design reusable and extensible programs (constructors in derived classes, member classes, pointers to objects, the this pointer and pointers to derived classes)

Nature of Assignment	:	Individual
Duration: One	Week	
Total Marks	:	50
Submission Date	:11/11/2026	

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment4 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192521288/SLOT-C-OOP.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to design a class hierarchy before writing its code, so that common behaviour is written once and new kinds of objects can be added without changing working classes.

You are expected to demonstrate the following progression:

> Interpret the Requirement → Identify Is-a and Has-a Relationships → Design the Hierarchy → Decide Access → Implement → Trace Construction → Challenge → Refine

Where pseudocode is required, it must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode. All code must compile with g++ -std=c++17 -Wall -Wextra.

TASK 1 – CLASS DESIGN REVIEW (6 Marks)

1.1 Diagnose the original design

- Professor and LabAssistant repeat name, id, basic and a pay rule. Repeated implementation can drift when a policy changes.
- All data members are public, so invalid values can be assigned without validation and internal representation is exposed.
- Equivalent concepts use inconsistent names: name versus nm, and id versus code.
- The da member is declared in Professor and LabAssistant but is not used in either pay() calculation.
- The value 0.42 is a magic number with no named policy or explanation. A future allowance change requires editing multiple methods.
- GuestFaculty has a different payment basis (hours × rate), but there is no shared interface for processing all staff uniformly.
- Adding a staff category requires creating another unrelated class and changing client code that knows each concrete type.
- All three classes expose different data layouts, making generic payroll reporting and maintenance harder.

1.2 Repaired hierarchy

Use Staff as the common abstract base class. Keep common identity and basic salary private/protected and expose controlled accessors. Each derived type overrides pay(). The DA policy is represented by a named constant and the unused da field is removed unless the college confirms that it is a separate allowance.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Staff {
protected:
    std::string name;
    int id;
public:
    Staff(std::string n, int i) : name(std::move(n)), id(i) {}
    virtual double pay() const = 0;
    virtual void print() const {
        std::cout << id << " " << name << " " << pay() << '\n';
    }
    virtual ~Staff() = default;
};

class Professor : public Staff {
    double basic, research;
    static constexpr double DA_RATE = 0.42;
public:
    Professor(std::string n, int i, double b, double r)
        : Staff(std::move(n), i), basic(b), research(r) {}
    double pay() const override { return basic * (1.0 + DA_RATE) + research; }
};

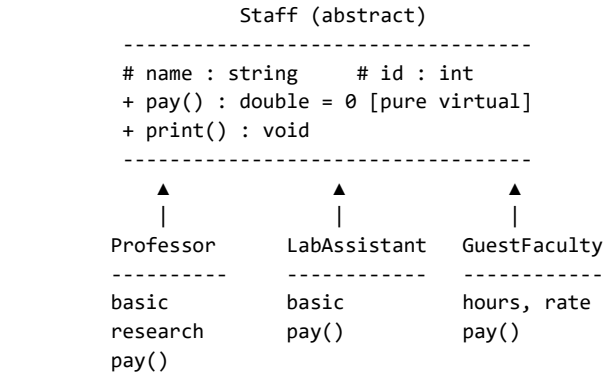
class LabAssistant : public Staff {
```

```
    double basic;
    static constexpr double DA_RATE = 0.42;
public:
    LabAssistant(std::string n, int i, double b)
        : Staff(std::move(n), i), basic(b) {}
    double pay() const override { return basic * (1.0 + DA_RATE); }
};

class GuestFaculty : public Staff {
    double hours, rate;
public:
    GuestFaculty(std::string n, int i, double h, double r)
        : Staff(std::move(n), i), hours(h), rate(r) {}
    double pay() const override { return hours * rate; }
};

int main() {
    std::vector<std::unique_ptr<Staff>> payroll;
    payroll.push_back(std::make_unique<Professor>("Meena", 101, 50000, 8000));
    payroll.push_back(std::make_unique<LabAssistant>("Arun", 102, 30000));
    payroll.push_back(std::make_unique<GuestFaculty>("Ravi", 103, 20, 1500));
    for (const auto& person : payroll) person->print();
}
```

UML-style class diagram



1.3 Design defence (under 100 words)

The hierarchy stores common identity and the pay() interface once in Staff. Each staff category supplies its own pay() implementation, so payroll code can process every employee through a Staff pointer in one loop. Adding a new category requires a new derived class and its pay() override, rather than changing a central switch statement and several existing classes. Encapsulation also protects employee data and named constants make policy values easier to maintain.

Alternative: one Staff class with a type and switch

```
enum class StaffType { Professor, LabAssistant, GuestFaculty };
struct StaffRecord {
    StaffType type;
    double basic, research, hours, rate;
    double pay() const {
        switch (type) {
        case StaffType::Professor: return basic * 1.42 + research;
        case StaffType::LabAssistant: return basic * 1.42;
        case StaffType::GuestFaculty: return hours * rate;
        }
        return 0.0;
    }
```

```
};
```

Both approaches perform O(1) work per pay() call. The single-class design may reserve fields that a particular staff category does not use, wasting per-object space. Adding a category requires changing the enum, the switch, field/validation logic and often client code. With polymorphism, the common interface remains stable and the new derived type can be added separately.

TASK 2 – FROM REQUIREMENT TO HIERARCHY (8 Marks)

2.1 Relationship classification

Relationship	Classification	Inheritance type
Student is a Person	Is-a; public inheritance	Single
Staff is a Person	Is-a; public inheritance	Single
TeachingAssistant is a Student and Staff	Is-a from both bases; public inheritance	Multiple; with shared virtual Person it forms a hybrid diamond
ResearchScholar is a Student	Is-a; public inheritance	Multilevel / hierarchical branch
Person has an Address/contact details object (if modelled separately)	Has-a; member object	Composition, not inheritance

Student and Staff are sibling derived classes of Person (hierarchical inheritance). TeachingAssistant inherits from both Student and Staff (multiple inheritance). ResearchScholar derives from Student (multilevel chain Person → Student → ResearchScholar). Together these form a hybrid inheritance design.

2.2 Diamond problem and virtual base

Without virtual inheritance, TeachingAssistant contains one Person subobject through Student and another through Staff. Access to a shared member such as name becomes ambiguous, and duplicate Person state is stored. Declaring both Student and Staff as virtual public bases of Person ensures the most-derived TeachingAssistant contains one shared Person subobject.

2.3 Class declarations

```
#include <string>
#include <utility>

class Person {
protected:
    std::string name;
    int id;
    std::string phone;
public:
    Person(std::string n, int i, std::string p)
        : name(std::move(n)), id(i), phone(std::move(p)) {}
};

class Student : virtual public Person {
protected:
    std::string programme;
    double cgpa;
public:
    Student(std::string n, int i, std::string p,
            std::string prog, double g)
        : Person(std::move(n), i, std::move(p)),
          programme(std::move(prog)), cgpa(g) {}
};

class Staff : virtual public Person {
protected:
```

```
    std::string department;
    double salary;
public:
    Staff(std::string n, int i, std::string p,
          std::string dept, double s)
        : Person(std::move(n), i, std::move(p)),
          department(std::move(dept)), salary(s) {}
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(std::string n, int i, std::string p,
                      std::string prog, double g,
                      std::string dept, double s)
        : Person(n, i, p),
          Student(n, i, p, prog, g),
          Staff(n, i, p, dept, s) {}
};

class ResearchScholar : public Student {
    double fellowship;
public:
    ResearchScholar(std::string n, int i, std::string p,
                    std::string prog, double g, double f)
        : Person(std::move(n), i, std::move(p)),
          Student(n, i, p, std::move(prog), g), fellowship(f) {}
};
```

In the most-derived class TeachingAssistant, the virtual Person base is initialised directly. Initialisers for Person written inside Student and Staff are ignored when those classes are constructed as base subobjects of TeachingAssistant. If Student or Staff is itself most-derived, its own Person initialiser is used.

2.4 Access table: Person member as seen in Student

Person member access	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not directly accessible	not directly accessible	not directly accessible

Predict the supplied program

Predicted output:

```
Person Student Staff TA
```

Person is printed once because Person is a virtual base shared by Student and Staff. If virtual is removed from both declarations, the predicted output is:

```
Person Student Person Staff TA
```

There are then two Person subobjects. This simple program is still valid because it does not access an ambiguous Person member. A compiler need not issue an error merely because the diamond exists; ambiguity is diagnosed when an ambiguous member or conversion is used.

Alternative: composition

With composition, TeachingAssistant contains a Student member and a Staff member. Each member contains its own Person subobject, so the complete object has two Person states. Access must be qualified, such as ta.student.name and ta.staff.name (subject to accessors). Composition avoids the

inheritance diamond and can make the roles explicit, but it is a has-a relationship rather than the is-a substitutability of a TeachingAssistant as a Student or Staff. Compare sizeof(TeachingAssistant) for both designs on the same compiler; exact sizes are implementation-dependent. Member access is O(1) in either design.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES (9 Marks)

3.1 Base constructor arguments and virtual base initialisation

The Person constructor receives name, ID and phone. Student passes its values to Person and initialises its own programme/CGPA. Staff passes values to Person and initialises department/salary. TeachingAssistant explicitly initialises the shared virtual Person base because it is the most-derived class.

3.2 Address and Course member classes

```
#include <iostream>
#include <string>
#include <utility>

class Address {
    std::string city;
    int pin;
public:
    Address(std::string c, int p) : city(std::move(c)), pin(p) {
        std::cout << "Address+ ";
    }
    ~Address() { std::cout << "Address- "; }
    std::string str() const { return city + "-" + std::to_string(pin); }
};

class Course {
    std::string title;
public:
    explicit Course(std::string t) : title(std::move(t)) {
        std::cout << "Course+ ";
    }
    ~Course() { std::cout << "Course- "; }
};

class Person {
protected:
    std::string name;
    int age;
    Address address;
public:
    Person(std::string n, int a, std::string city, int pin)
        : name(std::move(n)), age(a), address(std::move(city), pin) {
        std::cout << "Person+ ";
    }
    virtual ~Person() { std::cout << "Person- "; }
};

class Student : public Person {
    long long regNo;
    Course course;
public:
    Student(std::string n, int a, long long r, std::string city,
            int pin, std::string courseName)
        : Person(std::move(n), a, std::move(city), pin),
          regNo(r), course(std::move(courseName)) {
        std::cout << "Student+ ";
    }
    ~Student() override { std::cout << "Student- "; }
};
```

A member object is constructed automatically as part of its containing object. Its constructor arguments are supplied in the containing class's member-initializer list. It is destroyed automatically after the containing class destructor body finishes.

3.3 Construction/destruction trace

For the supplied Car program, the predicted output is:

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Construction order: direct base classes first, then member objects in declaration order, then the derived-class constructor body. Destruction is the reverse: derived destructor body, member objects in reverse declaration order, then base classes in reverse construction order.

For the expanded Person/Student example above, construction is Address+, Person+, Course+, Student+; destruction is Student-, Course-, Person-, Address-. Address is a member of Person and Course is a member of Student.

3.4 Virtual inheritance experiment

Remove virtual from the Person base declarations and construct TeachingAssistant. Two Person constructor messages will appear instead of one, and the object contains two Person subobjects. sizeof(TeachingAssistant) will generally increase, but the exact byte difference depends on compiler, ABI, alignment and virtual-base bookkeeping. Compile both versions and record sizeof values; do not assume a universal numeric size.

Complete the Code – TODO 1

```
Student(const std::string& n, int a, long long r,
        const std::string& city, int p)
    : Person(n, a), regNo(r), home(city, p) {}
```

Placed in the supplied Student class, the completed class fragment is:

```
class Student : public Person {
    long long regNo;
    Address home;
public:
    Student(const std::string& n, int a, long long r,
            const std::string& city, int p)
        : Person(n, a), regNo(r), home(city, p) {}

    void print() const {
        std::cout << regNo << ' ' << name << " (" << age << ") "
                  << home.str() << '\n';
    }
};
```

Expected output:

```
192311001 Kavya (19) Salem-636001
```

The actual member declaration order is regNo followed by home, so that is also the correct initializer-list order. To trigger -Wreorder, temporarily write home(city,p), regNo(r) while leaving the declarations unchanged. The compiler warns that home is listed before regNo but regNo is initialised first.

Alternative: dynamically allocated Address pointer

```
class Student {
    Address* home;
public:
```

```
    Student(const std::string& city, int pin)
        : home(new Address(city, pin)) {}
    Student(const Student& other)
        : home(new Address(*other.home)) {} // deep copy
    Student& operator=(const Student& other) {
        if (this != &other) {
            Address* fresh = new Address(*other.home);
            delete home;
            home = fresh;
        }
        return *this;
    }
    ~Student() { delete home; }
};
```

A member object requires no separate heap allocation and is automatically copied/destroyed. A pointer representation stores one pointer in the Student object and incurs a separate allocation/deallocation (typically O(1) for a fixed-size Address). The class must define ownership: constructor allocates; copy constructor deep-copies; copy assignment guards self-assignment and replaces safely; destructor deletes. Alternatively use std::unique_ptr<Address> and explicitly choose whether copying should be disabled or implemented as a deep copy.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS (7 Marks)

4.1 Identify the defects

- rate() in Account is non-virtual, so Account* calls Account::rate() using static binding. SeniorAccount::rate() does not override polymorphically.
- Account has a non-virtual destructor. Deleting a SeniorAccount object through Account* is undefined behaviour; the derived destructor may not run, so history is not released correctly.
- Account copy = SeniorAccount() copies only the Account base subobject. This is object slicing: the SeniorAccount-specific part is discarded.
- Storing derived objects by value in vector<Account> also slices them. Use pointers or references to preserve dynamic type.

4.2 Sanitizer/destructor evidence

Add messages to both destructors and run with AddressSanitizer. With a non-virtual base destructor, deleting through Account* is undefined behaviour; a typical run may show only the base destructor and leak the SeniorAccount history allocation. The exact sanitizer diagnostic depends on compiler/runtime. Save the real output.

4.3 Repaired polymorphic program

```
#include <iostream>
#include <memory>
#include <vector>

class Account {
public:
    virtual double rate() const { return 3.5; }
    virtual ~Account() { std::cout << "Account destroyed\n"; }
};

class SeniorAccount : public Account {
    double* history;
public:
    SeniorAccount() : history(new double[100]{}) {}
    double rate() const override { return 4.0; }
    ~SeniorAccount() override {
        delete[] history;
        std::cout << "SeniorAccount destroyed\n";
    }
};

int main() {
    std::vector<std::unique_ptr<Account>> accounts;
    accounts.push_back(std::make_unique<Account>());
    accounts.push_back(std::make_unique<SeniorAccount>());

    for (const auto& a : accounts)
        std::cout << a->rate() << '\n';
}
```

Expected rate output is 3.5 followed by 4. The unique_ptr objects automatically delete their Account-derived objects when the vector is destroyed. Because Account has a virtual destructor, the derived destructor is called.

4.4 Prevention rules

- If a base class is intended to be deleted through a base pointer or has virtual functions, give it a public virtual destructor (or a protected non-virtual destructor if deletion through the base is intentionally forbidden).
- Use override on every intended override so the compiler checks the signature and virtual dispatch relationship.
- Use owning smart pointers or clear ownership rules; avoid storing polymorphic derived objects by value in vector<Base>.

Predict the supplied program

Predicted output:

```
3.5 4 3.5
```

p->rate() calls the non-virtual Account::rate() because the static type of p is Account*. p->vrate() dispatches at run time and calls SeniorAccount::vrate(), producing 4. copy is a separate Account object initialised from the Account base portion of s; slicing removes the derived part, so copy.vrate() calls Account::vrate() and prints 3.5.

Alternative: vector<unique_ptr<Account>>

Use std::vector<std::unique_ptr<Account>> and add make_unique<Account>() or make_unique<SeniorAccount>(). Processing n accounts remains O(n) total and O(1) per account. The vector stores one smart pointer per entry plus each allocated object. RAII prevents forgotten manual delete and the virtual destructor ensures correct derived cleanup. Static dispatch remains a possible defect if rate() is not virtual; object slicing is avoided because the vector stores pointers, not Account values.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER (7 Marks)

5.1 Four ways to reach a Cart

```
#include <iostream>
#include <string>
#include <iomanip>

class Cart {
    std::string items[10];
    double prices[10]{};
    int n = 0;
    double discount = 0;
public:
    Cart& add(const std::string& item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            ++n;
        }
        return *this;
    }
    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }
    double total() const {
        double t = 0;
        for (int i = 0; i < n; ++i) t += prices[i];
        return t * (100 - discount) / 100;
    }
};

int main() {
    Cart direct;                        // (a) direct object
    direct.add("Rice", 60);

    Cart* ptr = &direct;                // (b) pointer, non-owning
    ptr->add("Dal", 120);

    Cart& ref = direct;                 // (c) reference alias
    ref.setDiscount(10);

    Cart* dynamic = new Cart;           // (d) dynamic object
    dynamic->add("Oil", 180);
    std::cout << std::fixed << std::setprecision(2)
              << direct.total() << ' ' << dynamic->total() << '\n';
    delete dynamic;
}
```

The four forms are: direct object access with dot; pointer access with arrow; reference access with dot; and a dynamically allocated object accessed through a pointer and released with delete. A reference is an alias and must be initialised; a non-owning pointer can be null and must be checked before dereference. A dynamically allocated object remains alive until released, so RAII/smart pointers are safer in modern C++.

Complete the Code – TODO 1 and TODO 2

```
Cart& add(const std::string& item, double price) {
    if (n < 10) {
        items[n] = item;
        prices[n] = price;
```

```
            ++n;
        }
        return *this;
    }

    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }
```

The supplied main() should produce:

```
324.00
```

The cart subtotal is 60 + 120 + 180 = 360. A 10% discount gives 360 × 0.90 = 324.

5.2 Find the cart with the highest total using pointers

```
Cart c1, c2, c3;
c1.add("A", 100);
c2.add("B", 250);
c3.add("C", 180);
Cart* carts[] = { &c1, &c2, &c3 };

Cart* highest = carts[0];
for (int i = 1; i < 3; ++i) {
    if (carts[i]->total() > highest->total())
        highest = carts[i];
}
// highest points to c2 in this example.
```

The search examines each cart once: O(n) time and O(1) extra space.

5.3 Meaning of this and comparison

Inside a non-static member function, this is a pointer to the object on which the function was called. *this is the actual current object as an lvalue. Returning *this by reference allows chained calls without copying the Cart. In setDiscount, this->discount identifies the data member while discount alone names the parameter.

Access method	Lifetime / safety	Typical use
Direct object	Automatic lifetime follows scope; safest simple ownership	Local objects with known scope
Pointer with ->	May be null; does not own object unless explicitly designed to	Optional access, arrays, polymorphism
Reference	Alias cannot normally be reseated or null	Required existing object, parameter passing
new/delete pointer	Manual lifetime; leaks possible if delete is missed	Dynamic lifetime; prefer smart pointers

If add() returned Cart by value instead of Cart&, each call would return a copy. Chaining would continue on temporary copies rather than the original object in the intended way, and copying would add overhead. Returning Cart& modifies and returns the same object.

Alternative: vector<Cart> rather than array of pointers

The maximum-total search remains O(n) for n carts. vector<Cart> stores objects contiguously and avoids one separate allocation per Cart, but vector growth may move/reallocate its elements and invalidate pointers, references and iterators into the vector. An array of pointers stores one pointer per entry and can point to separately managed objects; the pointed-to objects do not move merely because the pointer array changes. The pointer version requires explicit lifetime ownership. Reserve capacity or use stable ownership containers when references must remain valid.

TASK 6 – DESIGN CHALLENGE: VEHICLE RENTAL FLEET (13 Marks)

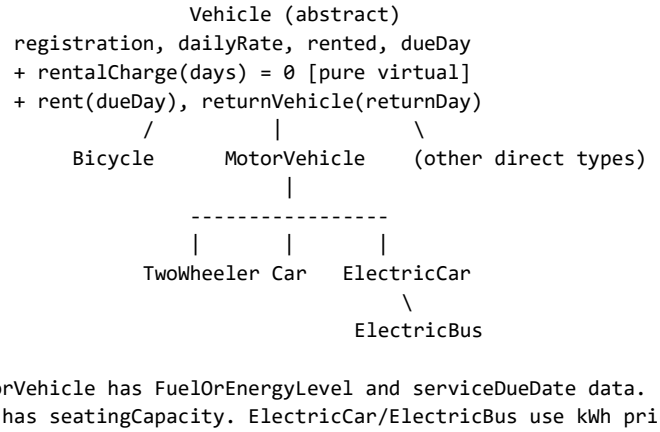
Stage 1 – Interpret the problem

Entities: Vehicle (common registration and daily rate), Bicycle, TwoWheeler, Car, ElectricCar and a future ElectricBus. Motor vehicles additionally have a fuel/charge level and service due date. Cars have seating capacity. Electric vehicles use energy charged per kWh rather than fuel per litre.

Testable requirement	Acceptance condition
Common identity	Every vehicle stores and displays registration number and daily rate.
Different rental charges	Each concrete vehicle calculates charge using its own pricing rule.
Rent/return state	A vehicle cannot be rented twice; returning an un-rented vehicle is rejected.
Late return	Return after the due day adds a late charge.
Polymorphic listing	The fleet lists all concrete vehicles through base-class pointers.
Extensibility	A new ElectricBus class can be added without editing existing vehicle classes or the fleet-processing loop.

Pricing assumptions used for this demonstration (the assignment does not specify actual rates): Bicycle charge = daily rate × days; TwoWheeler = daily rate × days + fuel usage × per-litre rate; Car = daily rate × days + fixed cleaning fee; ElectricCar = daily rate × days + kWh consumed × per-kWh rate; ElectricBus = daily rate × days + kWh consumed × per-kWh rate + fixed bus service fee. Replace these assumptions with faculty-approved rules if provided.

Stage 2 – Proposed hierarchy



Inheritance: Bicycle and MotorVehicle are public derived classes of Vehicle (hierarchical inheritance). TwoWheeler, Car and ElectricCar derive from MotorVehicle (multilevel/hierarchical). ElectricBus derives from ElectricCar (multilevel). Fuel/service data are held in MotorVehicle; seating capacity is held in Car; an optional ServiceDate class could be a member object.

Stage 3 – Critical design decisions

- Vehicle is abstract because rentalCharge(int days) is pure virtual. Common state and rent/return operations are stored once.
- rentalCharge() is virtual and overridden in every concrete type. display() may be virtual to list category-specific information. The destructor is virtual.
- Store the fleet as an array or vector of Vehicle pointers so the concrete type is preserved and a single loop can call virtual functions.
- Use unique_ptr<Vehicle> for ownership in production code; it is still a collection of base-class pointers and automatically releases objects.
- Use integer day counts and explicit late-day calculations. A production application should use calendar dates and validated money/energy units.

Stage 4 – Class declarations and pseudocode

Core interface:

```
class Vehicle {
protected:
    std::string registration;
    double dailyRate;
    bool rented = false;
    int dueDay = 0;
public:
    Vehicle(std::string reg, double rate);
    virtual double rentalCharge(int days) const = 0;
    bool rent(int returnDueDay);
    double returnVehicle(int actualDay, double lateFeePerDay);
    virtual void display() const;
    virtual ~Vehicle() = default;
};
```

SIMATS-style algorithmic pseudocode – RENT VEHICLE:

```
ALGORITHM RentVehicle(vehicle, dueDay)
BEGIN
    IF vehicle is NULL THEN
        PRINT "Invalid vehicle"
        RETURN FALSE
    ENDIF
    IF vehicle.rented = TRUE THEN
        PRINT "Vehicle already rented"
        RETURN FALSE
    ENDIF
    vehicle.rented ← TRUE
    vehicle.dueDay ← dueDay
    PRINT "Rental confirmed"
    RETURN TRUE
END
```

SIMATS-style algorithmic pseudocode – RETURN VEHICLE:

```
ALGORITHM ReturnVehicle(vehicle, actualDay, lateFeePerDay)
BEGIN
    IF vehicle is NULL OR vehicle.rented = FALSE THEN
        PRINT "Vehicle is not currently rented"
        RETURN -1
    ENDIF
    lateDays ← MAX(0, actualDay - vehicle.dueDay)
    baseCharge ← vehicle.rentalCharge(MAX(1, vehicle.dueDay - rentalStartDay))
    lateCharge ← lateDays × lateFeePerDay
```

```
    vehicle.rented ← FALSE
    PRINT baseCharge + lateCharge
    RETURN baseCharge + lateCharge
END
```

For the compact demonstration implementation below, returnVehicle receives the base rental amount already calculated for the booked duration and adds late days. A full production version would store the rental start day as well.

Stage 5 – Complete demonstration program

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
#include <algorithm>

class Vehicle {
protected:
    std::string reg;
    double dailyRate;
    bool rented = false;
    int dueDay = 0;
public:
    Vehicle(std::string r, double rate)
        : reg(std::move(r)), dailyRate(rate) {}
    virtual double rentalCharge(int days) const = 0;
    bool rent(int due) {
        if (rented || due < 0) return false;
        rented = true; dueDay = due; return true;
    }
    double returnVehicle(int actualDay, double lateFeePerDay) {
        if (!rented) return -1.0;
        int lateDays = std::max(0, actualDay - dueDay);
        rented = false;
        return lateDays * lateFeePerDay;
    }
    virtual void display() const {
        std::cout << reg << " daily rate=" << dailyRate
                  << " rented=" << std::boolalpha << rented << '\n';
    }
    virtual ~Vehicle() = default;
};

class Bicycle : public Vehicle {
public:
    using Vehicle::Vehicle;
    double rentalCharge(int days) const override {
        return dailyRate * days;
    }
};

class MotorVehicle : public Vehicle {
protected:
    double fuelOrChargeLevel;
    std::string serviceDueDate;
public:
    MotorVehicle(std::string r, double rate, double level, std::string date)
        : Vehicle(std::move(r), rate), fuelOrChargeLevel(level),
          serviceDueDate(std::move(date)) {}
};

class TwoWheeler : public MotorVehicle {
    double litresUsed, perLitre;
```

```
public:
    TwoWheeler(std::string r, double rate, double level, std::string date,
               double litres, double fuelRate)
        : MotorVehicle(std::move(r), rate, level, std::move(date)),
          litresUsed(litres), perLitre(fuelRate) {}
    double rentalCharge(int days) const override {
        return dailyRate * days + litresUsed * perLitre;
    }
};

class Car : public MotorVehicle {
protected:
    int seats;
    double cleaningFee;
public:
    Car(std::string r, double rate, double level, std::string date,
        int capacity, double clean)
        : MotorVehicle(std::move(r), rate, level, std::move(date)),
          seats(capacity), cleaningFee(clean) {}
    double rentalCharge(int days) const override {
        return dailyRate * days + cleaningFee;
    }
};

class ElectricCar : public Car {
protected:
    double kWhUsed, perKWh;
public:
    ElectricCar(std::string r, double rate, double level, std::string date,
                int capacity, double clean, double kWh, double energyRate)
        : Car(std::move(r), rate, level, std::move(date), capacity, clean),
          kWhUsed(kWh), perKWh(energyRate) {}
    double rentalCharge(int days) const override {
        return dailyRate * days + kWhUsed * perKWh + cleaningFee;
    }
};

// New type: added without modifying Vehicle or the fleet loop.
class ElectricBus : public ElectricCar {
    double busServiceFee;
public:
    ElectricBus(std::string r, double rate, double level, std::string date,
                int capacity, double clean, double kWh, double energyRate,
                double serviceFee)
        : ElectricCar(std::move(r), rate, level, std::move(date), capacity,
                      clean, kWh, energyRate), busServiceFee(serviceFee) {}
    double rentalCharge(int days) const override {
        return ElectricCar::rentalCharge(days) + busServiceFee;
    }
};

int main() {
    std::vector<std::unique_ptr<Vehicle>> fleet;
    fleet.push_back(std::make_unique<Bicycle>("BI-01", 100));
    fleet.push_back(std::make_unique<TwoWheeler>(
        "TW-02", 300, 70, "2026-12-01", 2, 110));
    fleet.push_back(std::make_unique<Car>(
        "CA-03", 1500, 60, "2026-12-10", 5, 250));
    fleet.push_back(std::make_unique<ElectricCar>(
        "EV-04", 1800, 85, "2026-12-15", 5, 200, 30, 8));
    fleet.push_back(std::make_unique<ElectricBus>(
        "EB-05", 4000, 90, "2026-12-20", 30, 500, 80, 9, 750));

    for (const auto& v : fleet) v->display();
```

```
    const int days = 2;
    for (const auto& v : fleet) {
        if (v->rent(10))
            std::cout << "Rental " << v->rentalCharge(days) << '\n';
    }

    // All vehicles are due on day 10 and returned on day 12:
    // late charge = 2 days * Rs. 100/day.
    for (const auto& v : fleet) {
        double late = v->returnVehicle(12, 100);
        std::cout << "Late charge " << late << '\n';
    }
}
```

The program demonstrates five concrete types, including ElectricBus. The fleet loop does not need to know the concrete type; it calls rentalCharge() through Vehicle. Every vehicle is rented, the charge is computed polymorphically, and a two-day late return produces a late charge of Rs. 200 per vehicle under the demonstration policy.

Alternative: one Vehicle class with a type code and switch

A single class would store a type enum and a union or a set of fields for all possible vehicle categories; rentalCharge() would switch on the type and apply the matching formula. Both a switch and virtual dispatch are O(1) per call, so processing n vehicles is O(n). The single class may contain unused fields in every object. A polymorphic object typically carries a vptr (one implementation pointer) plus only the fields relevant to its class. Adding a new type requires editing the enum, switch, validation/field layout and potentially client logic. With an abstract base class, add a derived class and leave the common fleet loop unchanged.